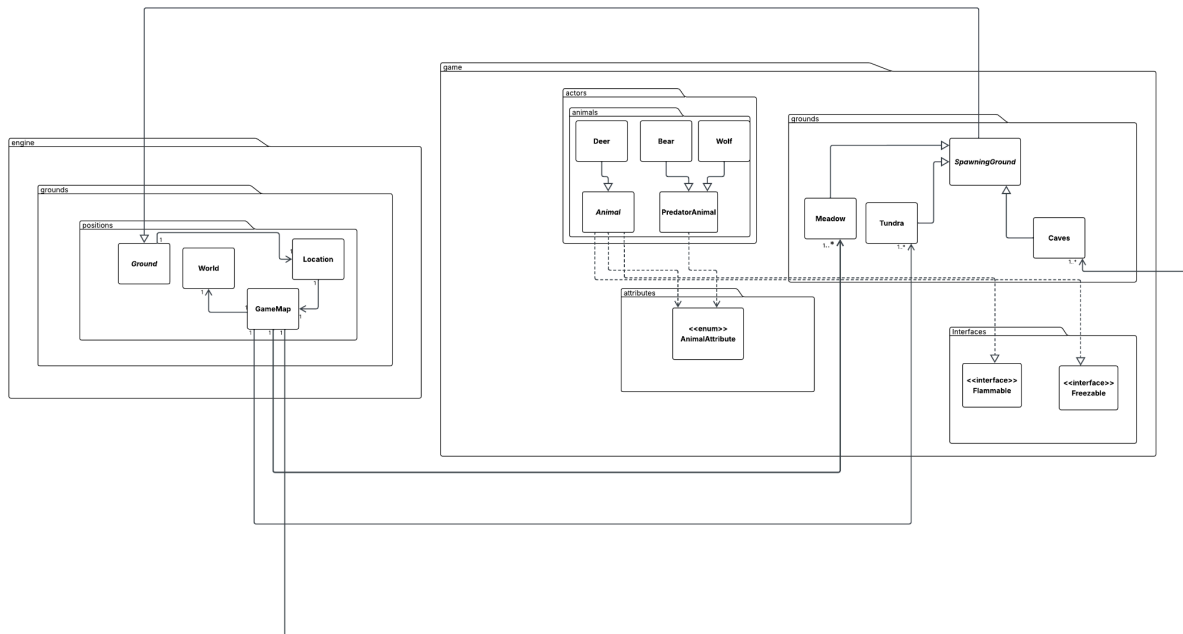


FIT2099 Assignment 2 Requirement 2

Design Rationale

Timothy Lim
33111472



REQ2 – Animal Spawning and Warmth System

Design Rationale (A2)

Design Overview

Problem & Goals (context)

- Automate animal population with tile-specific rules (Tundra/Cave/Meadow).
- Every Animal must have WARMTH_LEVEL that decrements -1/turn and despawns at 0.

- Tundra spawns: +10 max HP & heal to max; Cave: every 5 turns; Meadow: every 7 turns with 50% and grant CAN_CONSUME.
- Animals also participate in elemental effects via Flammable and Freezable.
- Keep the solution DRY, KISS, SOLID, and easy to extend/test.

Fixed Spawning Framework (applies to A/B/C)

- SpawningGround centralizes cadence & placement logic (cooldown, chance roll, equal-chance pick via `List<Supplier<? extends Actor>>`, safe placement to current/neighboring tile).
- Tundra/Cave/Meadow subclass SpawningGround and define their small policy overrides; local spawn effects are applied by wrapping suppliers.
- Per-tile configuration = pass different supplier lists to different tiles to meet Forest/Plains requirement

Design A — Constructor-Required Warmth in Animal

Idea: Add an `Animal(String name, char ch, int hp, int warmth)` ctor that installs `WARMTH_LEVEL`. Every subclass must call the warmth-aware ctor.

Advantages	Disadvantages
Compile-time explicitness: every animal must pass a warmth value.	Breaks existing subclass constructors (churn across code/tests).
No runtime checks needed for missing warmth.	Risk of accidentally using the legacy 3-arg ctor unless it's deprecated or removed.

Easy to read at call sites (species ctor shows its warmth).	Less flexible for “non-warmth” special cases (you’d need sentinel values or a different type).
---	--

Alternative Designs:

Design B — Abstract Hook in `Animal` (`startingWarmth()`)

Idea: `Animal`’s ctor calls protected abstract `int startingWarmth()` and installs `WARMTH_LEVEL` from that value.

Pros	Cons
Correctness by construction without changing call sites: <i>every</i> <code>Animal</code> must implement <code>startingWarmth()</code> .	Obliges each new species to add one method (tiny overhead).
SRP & OCP friendly: warmth policy lives in <code>Animal</code> species; adding new animals requires a tiny override only.	Warmth is <i>indirectly</i> visible (in override) rather than explicit ctor arg.
Backwards compatible with existing <code>Animal(name, ch, hp)</code> signatures (low churn).	If a non-warmth entity later extends <code>Animal</code> , you must still define a value (can be 0 if intended).

Design C — **Composition** via `WarmthComponent` (and optional global lifecycle)

Pros	Cons
Very modular: warmth logic isolated in one component, reusable and mockable.	More boilerplate (construct and attach the component everywhere).
Easiest to unit test in isolation.	Risk of forgetting to attach the component on a new species.

If later you want non-animal actors with warmth, just inject the component.	Indirection makes the simple rule (-1/turn; remove at 0) harder to trace through code.
	Adds moving parts without a clear requirement need right now.

Final Design Choice and Justification

Chosen Design:

Abstract `SpawningGround` base class with unified logic and three concrete subclasses (`Tundra`, `Cave`, `Meadow`), each defining its spawning probability, cooldown, and animal-specific effects. Warmth mechanics are integrated into the `Animal` hierarchy.

Why This Design Was Chosen:

Reason	Justification
Matches specification perfectly	Each ground type spawns animals under distinct rules with per-map customisation.
DRY & KISS adherence	The shared spawning algorithm removes redundancy and simplifies code maintenance.
Strong SOLID compliance	New spawner or animal types require zero changes to existing code; policies remain localised.
Low connascence	<code>Tundra/Cave/Meadow</code> logic remains self-contained; effects don't leak into other classes.
Extensibility & scalability	Adding new environments (e.g., <code>Desert</code> , <code>Jungle</code>) or animals only needs minimal subclassing and

	configuration.
Ease of testing & debugging	Deterministic RNG support allows unit tests for spawn cadence and probabilities.
Future Readiness	Easily extendable to global spawn balancing or analytics by adding observers later if needed.

6) Elemental Interfaces (applies across designs)

- Animals implement `Flammable` & `Freezable` so `Burning` and `Frozen` systems can apply consistently.
- This remains orthogonal to warmth: burning might reduce HP over time; freezing might block actions or slow movement; warmth continues to tick each turn and drives despawn at 0.
- Coexistence is simple and testable: ensure that both warmth and elemental effects advance under the same engine tick.

7) Pros & Cons of the Overall (current) architecture

(Spawning framework + Design B for warmth + elemental interfaces)

Pros (for table):

- DRY: One spawn engine; one warmth tick path. Local tundra/meadow effects stay where they belong.
- KISS: Three small spawner overrides; one abstract warmth hook; plain interface flags for elements.
- SOLID:

- SRP: SpawningGround (cadence/placement), concrete grounds (policy/effects), Animal (lifecycle), species (values/behaviours).
- OCP: add new spawners/animals with tiny changes; no existing code edits.
- LSP: all grounds substitute for SpawningGround; all species substitute for Animal.
- ISP/DIP: rely on Supplier<? extends Actor> and small interfaces.
- Connascence localized: tundra bonus and meadow consume capability live in their own classes; warmth value lives in species; warmth ticking in Animal.
- Extensibility: per-tile spawn tables; repeat suppliers to weight chances; add behaviours/capabilities without touching the framework.
- Testability: RNG injection for deterministic cadence; warmth decrement asserts over turns; elemental effects test in isolation.

Cons (for table):

- Java 8 supplier type inference can be verbose (`Arrays.<Supplier<? extends Actor>>asList(...)`).
- Local wrap on spawn (for tundra/meadow effects) adds a tiny indirection.
- No global spawn analytics out-of-the-box (could add an observer later).
- Species authors must remember to implement `startingWarmth()` (by design; compile-time enforced).

Conclusion

8) Chosen Design & Justification (final)

Chosen: Design B – Abstract Hook in Animal (startingWarmth()), with:

- PredatorAnimal receiving its warmth in its ctor and returning it via startingWarmth().
- Deer overriding startingWarmth() to return 10.
- All Animals implement Flammable + Freezable for elemental compatibility.
- SpawningGround framework as described (Tundra/Cave/Meadow policies + per-tile suppliers + local effects).

Why this pick (summary to paste):

- Meets the REQ2 wording unambiguously: all animals have warmth and are removed at 0; distinct spawner rules & effects satisfied; per-tile species mixes supported.
- Minimal churn / high safety: no broad constructor changes, small targeted overrides in species only.
- Strong design hygiene: DRY core spawning, KISS warmth enforcement, SOLID separation (policy vs. lifecycle), and localized connascence.
- Future-proof enough: easy to add animals/spawners; easy to integrate elemental rules; can introduce observers later if global analytics are needed.

One-line conclusion

This approach gives you correctness by construction for warmth, cleanly integrates elemental effects, keeps the spawning system DRY and extensible, and delivers exactly what REQ2 asks for with the least risk and the best maintainability.